

Structures de contrôle

Le langage C# est un langage de programmation structuré. La structure d'un programme définit l'ordre d'exécution des instructions : condition, boucles, ...

Condition

Une instruction peut s'exécuter lorsqu'une condition est vraie. La syntaxe est la suivante :

```
if (expression)  
    instruction
```

expression définit la condition et doit être du type `bool`, c'est-à-dire qu'elle ne peut valoir que `true` (vrai) ou `false` (faux).

L'instruction n'est exécutée que si la condition est vraie (`true`).

Exemple :

```
if (a==10) Console.WriteLine("a vaut 10");
```

Plusieurs instructions

Pour exécuter plusieurs instructions si la condition est vraie, *instruction* peut être remplacé par un bloc d'instructions entre accolades :

```
{  
    instruction  
    instruction  
    ...  
}
```

Exemple :

```
if (a==10)  
{  
    Console.WriteLine("a vaut 10");  
    a=9;  
    Console.WriteLine("désormais a vaut 9");  
}
```

Sinon...

Il est possible d'exécuter des instructions quand une condition est vraie, et d'autres instructions quand elle est fausse.

La syntaxe est la suivante :

```
if (expression)
    instruction
else
    instruction
```

Exemples :

```
if (a==10) Console.WriteLine("a vaut 10");
else Console.WriteLine("a ne vaut pas 10");
if ((a==10)&&(b==11)) Console.WriteLine("a vaut 10 et b vaut 11");
else Console.WriteLine("a ne vaut pas 10 ou b ne vaut pas 11");
```

Conditions multiples

L'enchaînement des instructions conditionnelles est possible.

Exemple :

```
if (a==10)
    if (b==11) Console.WriteLine("a vaut 10 et b vaut 11");
    else Console.WriteLine("a vaut 10 mais b ne vaut pas 11");
else Console.WriteLine("a ne vaut pas 10");
```

Chaque instruction `else` correspond au `if` qui le précède, s'il n'a pas déjà de `else`. Cependant, pour clarifier le code ou résoudre certains cas ambigus (pas de `else` pour le second `if`, par exemple), il est préférable de mettre des accolades :

```
if (a==10)
{
    if (b==11) Console.WriteLine("a vaut 10 et b vaut 11");
    else Console.WriteLine("a vaut 10 mais b ne vaut pas 11");
}
else Console.WriteLine("a ne vaut pas 10");
```

Autre exemple :

```
if (a==10) Console.WriteLine("a vaut 10");
else if (a==11) Console.WriteLine("a vaut 11");
else Console.WriteLine("a ne vaut ni 10, ni 11");
```

Dans cet exemple, chaque instruction `if` n'est testée que si la précédente est fausse. Si le nombre de cas est important et que chaque condition teste une valeur pour la même expression, il est préférable d'utiliser l'instruction `switch`.

Tests de plusieurs cas

L'instruction `switch` permet de tester la valeur d'une expression avec plusieurs cas.

La syntaxe est la suivante :

```
switch(expression)
{
    cas :
    cas :
    ...
        instructions
        fin du cas

    cas :
    cas :
    ...
        instructions
        fin du cas

    ...
}
```

Où *cas* peut avoir l'une des deux syntaxes suivantes :

```
case constante
    Cas où l'expression vaut la constante spécifiée.
default
    Cas où l'expression ne correspond à aucun autre cas (le cas par défaut).
```

fin du cas est une instruction spéciale terminant le ou les cas. Hormis les instructions interrompant le cours de l'exécution (une instruction interrompant ou continuant une boucle, une instruction `throw` pour lancer une exception, une instruction `return`), il s'agit en général de l'une des deux instructions suivantes :

```
break;
    Fin du test, la prochaine instruction exécutée est celle située après l'accolade fermante de switch.
```

Exemple :

```
switch(a)
{
    case 10 :
        Console.WriteLine("a vaut 10");
        break;

    case 11 :
        Console.WriteLine("a vaut 11");
        break;

    default :
        Console.WriteLine("a ne vaut ni 10, ni 11");
        break;
}
```

La syntaxe permet de rassembler plusieurs cas ensemble pour exécuter les mêmes instructions :

```
switch(a)
{
case 2 :
case 3 :
case 5 :
    Console.WriteLine("a vaut 2, 3 ou 5");
    break;

case 10 :
case 11 :
    Console.WriteLine("a vaut 10 ou 11");
    break;

default :
    Console.WriteLine("a n'est pas dans la liste (2, 3, 5, 10, 11)");
    break;
}
```

Il est également possible de poursuivre le traitement avec les instructions d'un autre cas :

```
switch(a)
{
case 10 :
    Console.WriteLine("a vaut 10");
    goto case 11;

case 11 :
    Console.WriteLine("a vaut "+a);
    break;

default :
    Console.WriteLine("a ne vaut ni 10, ni 11");
    break;
}
```

Boucles

Une boucle permet d'exécuter plusieurs fois une ou des instructions tant qu'une condition est vraie.

Boucle while

La boucle `while` est la plus simple : tant que la condition est vraie, elle exécute l'instruction ou le bloc d'instructions spécifié.

Sa syntaxe est la suivante :

```
while ( expression )
    instruction_ou_bloc
```

Où *expression* est une expression du type `bool`, c'est-à-dire valant `true` (vrai) ou `false` (faux).

Exemple : rechercher une valeur spécifique dans un tableau d'entiers

```
int i = 0;

// tant que i est un indice correct et que la valeur 0 n'est pas trouvée
while ( ( i < tableau.Length ) && ( tableau[i] != 0 ) )
{
    Console.WriteLine(tableau[i]);
    i++;
}
```

N.B.: Si la condition est fausse dès le début, l'instruction ou le bloc n'est pas exécuté, car la condition est testée avant.

Boucle do...while

La boucle `do...while` ressemble à la précédente, excepté que la condition est testée après. C'est-à-dire que le bloc d'instructions est toujours exécuté au moins une fois.

La syntaxe est la suivante :

```
do
{
    instructions
}
while ( expression );
```

Où *expression* est une expression de type `bool`.

Exemple :

```
string fichier;
do
{
    Console.Write("Entrez un nom de fichier ou rien pour quitter : ");
    fichier=Console.ReadLine();
    if (fichier != "") TraiterFichier(fichier);
}
while ( fichier != "" );
```

Boucle for

La boucle `for` regroupe plusieurs phases de la boucle :

- L'initialisation, par exemple un indice dans un tableau,
- La condition, tant qu'elle est vraie la boucle continue,
- Les instructions à exécuter,
- L'instruction (ou les instructions) d'incrémentation, exécutée(s) juste avant de tester à nouveau la condition.

La syntaxe d'une boucle `for` est la suivante :

```
for ( initialisation ; condition ; incrémentation )
    instructions
```

Exemple : recherche d'une valeur nulle dans un tableau d'entiers

```
for ( int i=0 ; (i<tableau.Length)&&(tableau[i]!=0) ; i++ )  
    Console.WriteLine( tableau[i] );
```

Boucle foreach

La boucle `foreach` parcourt tous les éléments d'un objet implémentant l'interface `IEnumerable`. Il s'agit d'une version simplifiée de la boucle `for`.

La syntaxe est la suivante :

```
foreach ( type variable in objet_IEnumerable )  
    instructions
```

Durant la boucle, la variable *variable* (de type *type*) vaut successivement toutes les valeurs retournées par l'objet implémentant l'interface `IEnumerable`.

Les tableaux implémentent cette interface, et peuvent donc être utilisés dans une boucle `foreach`.

Exemple : afficher tous les éléments d'un tableau

```
string[] messages = { "Test", "d'une", "boucle foreach" };  
foreach ( string s in messages )  
    Console.WriteLine(s);
```

Interrompre une boucle

L'instruction `break` permet d'interrompre prématurément une boucle.

Syntaxe :

```
break;
```

Exemple :

```
for(int i = 0 ; i < 10 ; i++)  
{  
    Console.WriteLine( "i vaut " + i );  
    if (i==5) break; // Arrêter à 5  
}
```

Le code ci-dessus affiche :

```
i vaut 0  
i vaut 1  
i vaut 2  
i vaut 3  
i vaut 4  
i vaut 5
```

Continuer une boucle

L'instruction `continue` permet de poursuivre une boucle, c'est-à-dire passer immédiatement à l'itération suivante sans exécuter les autres instructions de la boucle.

Syntaxe :

```
continue;
```

Exemple :

```
for(int i = 0 ; i < 10 ; i++)
{
    if (i==5) continue; // Sauter 5
    Console.WriteLine( "i vaut " + i );
}
```

Le code ci-dessus affiche :

```
i vaut 0
i vaut 1
i vaut 2
i vaut 3
i vaut 4
i vaut 6
i vaut 7
i vaut 8
i vaut 9
```

L'interface `IEnumerable` et le mot clé `yield`

L'interface `IEnumerable` est définie dans deux espaces de nom différents :

- `System.Collections` : (Framework 1.0) utilise le type `object`,
- `System.Collections.Generic` : (Framework 2.0) utilise un [type générique](#).

Comme la plupart des classes de ces deux espaces de nom, ces deux classes ne diffèrent que par le type utilisé pour chaque élément : `object` ou générique. L'interface `IEnumerable` ne définit qu'une seule méthode :

- `System.Collections.IEnumerable`:

```
IEnumerator GetEnumerator();
```

- `System.Collections.Generic.IEnumerable`:

```
IEnumerator<T> GetEnumerator();
```

L'implémentation d'une fonction retournant une instance de l'interface `IEnumerator` est simplifiée en utilisant le mot clé `yield`. Celui-ci est utilisé pour retourner chaque élément de l'itération ou pour interrompre l'itération :

- L'instruction `yield return` est suivie de la valeur de l'élément à retourner,
- L'instruction `yield break` interrompt la boucle prématurément.

Exemple :

```
using System.Collections.Generic;

public class Mots
{
    // Tableau contenant des mots :
    public string[] mots;

    // Constructeur
    public Mots(params string[] mots)
    { this.mots = mots; }

    // Énumérer tous les mots jusqu'à trouver
    // le mot contenu dans le paramètre dernier (exclu)
    // ou la fin du tableau.
    public IEnumerable<string> TousLesMotsJusquA(string dernier)
    {
        foreach(string mot in mots)
            if (mot.Equals(dernier)) yield break; // fin prématurée de
l'itération
            else yield return mot; // élément de l'itération
        // fin normale de la boucle d'itération
    }
}

public class TestProgram
{
    public static void Main()
    {
        Mots fruits = new Mots( "pomme", "poire", "abricot", "fraise",
"kiwi" );
        foreach( string fruit in fruits.TousLesMotsJusquA("fraise") )
            Console.WriteLine( fruit );
    }
}
```

Ce programme affiche :

```
pomme
poire
abricot
```

Les fruits suivants ne sont pas affichés car le mot "fraise" a été trouvé.